

Guía para recrear un proyecto por capas

Proyecto de ejemplo: Sistema de Gestión de Biblioteca

Esta guía propone recrear un proyecto nuevo utilizando como referencia una arquitectura por capas, sin copiar el dominio original de profesores y estudiantes. No incluye código de implementación; indica carpetas, archivos, responsabilidades, propiedades de los modelos y conceptos de POO a practicar.

1. Arquitectura general

La aplicación se organizará por capas. La idea principal es que cada parte tenga una responsabilidad clara y que las dependencias se conecten mediante abstracciones.

App → Configuración de dependencias → Vista → Controlador → Servicio → Repositorio → Persistencia

Además, los modelos se transformarán en DTO mediante mapeadores.

Modelo ↔ Mapeador ↔ DTO

2. Estructura de carpetas

```
src/
  com/
  biblioteca/
  App
  AppScannerCli
  config/
    ConfiguracionDependencia
    RutaPersistencia
    modulo/
      ModuloConfigurable
      ConfiguracionModuloLibro
      ConfiguracionModuloAutor
      ConfiguracionModuloUsuario
      ConfiguracionModuloPrestamo
  controlador/
    LibroControlador
    AutorControlador
    UsuarioControlador
    PrestamoControlador
  dto/
    libro/
    autor/
    usuario/
    prestamo/
    validacion/
  mapeador/
  modelo/
    enumeracion/
    convertidor/
  persistencia/
  repositorio/
  cargador/
  servicio/
  vista/
  constante/
```

3. Modelos del dominio

Libro

Propiedad	Tipo	Descripción
idLibro	Long	Identificador
titulo	String	Título del libro
isbn	String	Código ISBN
anioPublicacion	Integer	Año de publicación
genero	String	Género literario
cantidadEjemplares	Integer	Cantidad de ejemplares
idAutor	Long	Identificador del autor
estado	EstadoEntidad	Activo/Inactivo

Autor

Propiedad	Tipo	Descripción
idAutor	Long	Identificador
nombre	String	Nombre
apellido	String	Apellido
nacionalidad	String	Nacionalidad
fechaNacimiento	LocalDate	Fecha de nacimiento
estado	EstadoEntidad	Activo/Inactivo

Usuario

Propiedad	Tipo	Descripción
idUsuario	Long	Identificador
nombre	String	Nombre
apellido	String	Apellido
documento	String	Documento de identificación
telefono	String	Teléfono
correo	String	Correo electrónico
estado	EstadoEntidad	Activo/Inactivo

Prestamo

Propiedad	Tipo	Descripción
idPrestamo	Long	Identificador
idUsuario	Long	Usuario que solicita
idLibro	Long	Libro prestado

Propiedad	Tipo	Descripción
fechaPrestamo	LocalDate	Fecha del préstamo
fechaDevolucionEsperada	LocalDate	Fecha límite
fechaDevolucionReal	LocalDate	Fecha de devolución
estado	EstadoPrestamo	Estado del préstamo

4. Enumeraciones

EstadoEntidad: ACTIVO e INACTIVO. Se utiliza para representar el estado general de las entidades.

EstadoPrestamo: PENDIENTE, DEVUELTO y ATRASADO. Se utiliza exclusivamente para el ciclo de vida de un préstamo.

Prestamo debe permitir una lógica de transición de estados. Por ejemplo: PENDIENTE → DEVUELTO o PENDIENTE → ATRASADO. La entidad debe controlar estas transiciones.

5. Responsabilidad de los modelos

Libro: Controlar su propio estado y permitir actualizar título, ISBN, género, cantidad de ejemplares, autor y estado.

Autor: Controlar nombre, apellido, nacionalidad, fecha de nacimiento y estado.

Usuario: Controlar nombre, apellido, documento, teléfono, correo y estado.

Prestamo: Controlar la devolución y las transiciones de estado.

Los modelos no deben mostrar menús, pedir datos por consola ni encargarse de guardar información en archivos. Su responsabilidad es representar y proteger el estado del dominio.

6. DTO

Libro

LibroCrearDto: titulo, isbn, anioPublicacion, genero, cantidadEjemplares, idAutor. LibroActualizarDto: idLibro y los campos que puedan modificarse. LibroDto: todos los datos relevantes, incluido estado.

Autor

AutorCrearDto: nombre, apellido, nacionalidad, fechaNacimiento. AutorActualizarDto: idAutor y campos modificables. AutorDto: todos los datos, incluido estado.

Usuario

UsuarioCrearDto: nombre, apellido, documento, telefono, correo. UsuarioActualizarDto: idUsuario y campos modificables. UsuarioDto: todos los datos, incluido estado.

Prestamo

PrestamoCrearDto: idUsuario e idLibro; el servicio puede calcular las fechas. PrestamoDevolucionDto: idPrestamo. PrestamoDto: idPrestamo, idUsuario, idLibro, fechas y estado.

7. Mapeadores

Crear una interfaz genérica Mapeador y mapeadores específicos: LibroMapeador, AutorMapeador, UsuarioMapeador y PrestamoMapeador. Su responsabilidad será transformar entidades del modelo en DTOs y, cuando sea necesario, realizar la conversión inversa.

Flujo: Modelo → Mapeador → DTO

8. Repositorios

Crear RepositorioBase con operaciones generales como guardar, actualizar, buscarPorId, eliminar, listarTodos, contar y cerrar.

Crear interfaces específicas: LibroRepositorio, AutorRepositorio, UsuarioRepositorio y PrestamoRepositorio.

Operaciones específicas que podrían agregarse posteriormente: LibroRepositorio puede buscar por ISBN, género o autor; UsuarioRepositorio puede buscar por documento; PrestamoRepositorio puede buscar préstamos por usuario, pendientes o atrasados.

9. Persistencia

Crear una RepositorioBaseAbstracto para reutilizar las operaciones comunes y luego las implementaciones LibroRepositorioImpl, AutorRepositorioImpl, UsuarioRepositorioImpl y PrestamoRepositorioImpl.

La persistencia debe quedar separada de la lógica de negocio. El servicio no debería saber los detalles de cómo se almacenan los datos.

10. Servicios

LibroServicio: Registrar, actualizar, buscar, listar y desactivar libros.

AutorServicio: Registrar, actualizar y listar autores.

UsuarioServicio: Registrar, actualizar, buscar y listar usuarios.

PrestamoServicio: Registrar préstamo, devolver libro, listar préstamos y consultar préstamos de un usuario.

PrestamoServicio será la parte con mayor lógica de negocio. Debe comprobar si el usuario existe y está activo, si el libro existe y está activo, si existen ejemplares disponibles y si hay condiciones que impidan el préstamo.

11. Controladores

Crear LibroControlador, AutorControlador, UsuarioControlador y PrestamoControlador. Su función será recibir las acciones de la vista y delegarlas al servicio correspondiente. No deben contener la lógica principal del negocio.

12. Vistas

Crear LibroVista, AutorVista, UsuarioVista y PrestamoVista. La vista se encarga exclusivamente de la interacción con el usuario de consola: mostrar menús, solicitar datos y mostrar resultados.

Menú principal sugerido: 1. Gestionar libros; 2. Gestionar autores; 3. Gestionar usuarios; 4. Gestionar préstamos; 0. Salir.

13. Configuración e inyección de dependencias

ConfiguracionDependencia será el punto central de composición. Cada ConfiguracionModulo... construirá las piezas de su módulo: repositorio, mapeador, servicio, controlador y vista.

La inyección será por constructor. Ejemplo conceptual: la vista recibe un controlador; el controlador recibe un servicio; el servicio recibe uno o varios repositorios y mapeadores. Así las clases no crean internamente todas sus dependencias.

14. Conceptos de POO que se practicarán

Concepto	Dónde practicarlo
Encapsulamiento	Atributos privados y métodos que controlan modificaciones en Libro, Autor, Usuario y Prestamo.
Abstracción	Interfaces de repositorios y mapeadores.
Herencia	RepositorioBaseAbstracto y sus implementaciones.
Polimorfismo	Referencias de tipo interfaz que apuntan a implementaciones concretas.
Interfaces	RepositorioBase, repositorios específicos y Mapeador.
Genéricos	RepositorioBase<T, ID> y Mapeador<E, R>.
Inyección de dependencias	Constructores de servicios, controladores y vistas.
Composición	ConfiguracionModulo... construyendo los objetos del módulo.
DTO	Objetos para transportar datos entre capas.
Enum	EstadoEntidad y EstadoPrestamo.
Máquina de estados	Transiciones controladas del estado de Prestamo.
Validación	ReglasValidacion.
Optional	Búsquedas que pueden no encontrar una entidad.
Caché	Cargadores con Map en memoria, si se decide mantener esta característica.
Builder	Configuración de la CLI, si la librería utilizada lo permite.
Separación de responsabilidades	Vista → Controlador → Servicio → Repositorio → Persistencia.

15. Orden recomendado de construcción

- FASE 1 — modelo: Libro, Autor, Usuario, Prestamo y enumeraciones.
- FASE 2 — dto: DTO de creación, actualización, devolución y salida.
- FASE 3 — mapeador: interfaz y mapeadores específicos.
- FASE 4 — repositorio: interfaces y cargadores.
- FASE 5 — persistencia: clase abstracta e implementaciones.
- FASE 6 — servicio: lógica de aplicación y reglas de negocio.
- FASE 7 — controlador: comunicación entre vista y servicio.
- FASE 8 — vista: menús e interacción con consola.
- FASE 9 — config: composición e inyección de dependencias.
- FASE 10 — App y AppScannerCli: arranque de la aplicación.

16. Flujo completo de una operación

Para registrar un libro, el usuario interactúa con la Vista. La Vista entrega los datos al Controlador. El Controlador delega al Servicio. El Servicio valida las reglas, crea o modifica la entidad y utiliza el Repositorio para persistirla. Finalmente, el Mapeador convierte la entidad en DTO y el resultado vuelve hacia la Vista.

Usuario → Vista → Controlador → Servicio → Entidad → Repositorio → Persistencia →
Entidad → Mapeador → DTO → Vista

17. Regla principal del proyecto

No copiar mecánicamente el proyecto anterior. Utilizarlo como referencia para entender qué problema resuelve cada capa. La meta es poder explicar por qué existe cada archivo y qué responsabilidad tiene, no solamente reproducir su estructura.